

Hierarchical Coarse-to-Fine RAG for Japanese Civil-Engineering Standards:

A Comparative Study of Naive, Hybrid, and Structure-Aware Retrieval

on River and Sediment Control Technical Documents

Takato Yasuno

Abstract

Answering precise technical questions against Japan’s multi-volume **River and Sediment Control Technical Standards** requires retrieving highly specific passages from a document corpus with rich hierarchical structure (volumes, chapters, and sections) under the strict constraint of a single consumer-grade GPU (16 GB VRAM). This paper presents a systematic comparison of three retrieval-augmented generation (RAG) strategies applied to this domain:

Naive RAG (flat dense vector search, FAISS IndexFlatIP), **Light RAG** (hybrid BM25 + dense fusion with $\alpha = 0.5$), and **HippoRAG2-Hier** (a knowledge-graph-free hierarchical coarse-to-fine strategy that exploits the Volume $\rightarrow$ Chapter $\rightarrow$ Chunk structure of the standards through cascaded representative-vector selection).

Each retrieval strategy is paired with two open-source Japanese LLMs served locally via Ollama—Swallow-8B-LoRA-Q4 and ELYZA-JP-8B-LoRA-Q4—yielding **six conditions** evaluated on a **200-question benchmark** (seed=42). Answers are scored automatically by Qwen2.5-7B-Instruct as AI-as-Judge on a 0–3 rubric measuring technical accuracy and standard citation quality.

Our results demonstrate that HippoRAG2-Hier consistently outperforms both baselines in judge average score and perfect-score rate while remaining within the 16 GB VRAM budget. The hierarchical pre-filtering reduces irrelevant chunk retrieval and concentrates context on technically relevant sections. Light RAG provides a stronger keyword-recall advantage than Naive RAG on definition-style questions but underperforms on procedure-centric questions where document structure matters more than term frequency. We also find that LLM choice interacts with retrieval strategy: ELYZA-JP-8B-LoRA

gains more from structural retrieval augmentation than Swallow-8B-LoRA, whose domain LoRA training partially compensates for weaker retrieval.

Keywords: Retrieval-Augmented Generation, Hierarchical RAG, BM25 Hybrid Retrieval, Japanese NLP, River Engineering, Technical Standards, LLM-as-Judge, Open-Source LLM, Domain Adaptation, Civil Engineering AI.

1 Introduction

Japan’s *River and Sediment Control Technical Standards* (*Kasen Sabo Gijutsu Kijun*), issued by the Ministry of Land, Infrastructure, Transport and Tourism (MLIT), span eight volumes covering survey methodology, planning procedures, structural design criteria, and maintenance protocols for river levees, dams, sabo (torrent-control) structures, and related facilities. Practitioners—field engineers, inspection officers, and dam operators—routinely require rapid, authoritative answers to highly specific questions embedded in these multi-hundred-page standards: inspection intervals, load calculation assumptions, maintenance-planning hierarchies, and hazard-response decision trees.

The retrieval challenge. Modern large language models (LLMs) trained on general corpora cannot reliably recall domain-specific regulatory values, section-level procedural constraints, or the precise hierarchy of Japanese civil-engineering standards. Retrieval-Augmented Generation (RAG) [6] addresses this by supplying retrieved passage context at inference time. However, *flat* dense vector retrieval—querying all chunks simultaneously—dilutes retrieval precision in hierarchically structured corpora: the top- k results may span multiple unre-

lated volumes, causing the LLM to generate hedged or off-topic responses.

Hierarchical structure as a retrieval prior.

The River and Sediment Control Technical Standards are organised in a natural three-level hierarchy: *Volume* (eight distinct technical areas) $\rightarrow$ *Chapter* (100+ chapters across all volumes) $\rightarrow$ *Chunk* (paragraph-level text segments of ≈ 500 characters). This structure provides a powerful coarse-to-fine retrieval prior: a question about sabo-dam design maintenance is unlikely to require context from the river-levee survey volume. HippoRAG [5] proposes graph-based hippocampal memory structures for LLM retrieval; our work adapts this spirit to a knowledge-graph-free hierarchical strategy that remains deployable under strict VRAM constraints.

Research questions. This paper addresses three questions:

1. Does hierarchical coarse-to-fine retrieval outperform flat dense retrieval on a highly structured Japanese regulatory corpus?
2. Does BM25-dense hybrid (Light RAG) provide complementary benefits to dense-only Naive RAG in this domain?
3. How do LLM choice and retrieval strategy interact in determining answer quality?

Contributions.

1. A reproducible six-condition benchmark comparing Naive RAG, Light RAG, and HippoRAG2-Hier on 200 questions from the River and Sediment Control Technical Standards, evaluated by AI-as-Judge scoring.
2. HippoRAG2-Hier: a knowledge-graph-free hierarchical RAG design that exploits document-structure metadata (Volume $\rightarrow$ Chapter $\rightarrow$ Chunk) via cascaded representative-vector selection, fitting within a 16 GB VRAM constraint.
3. An empirical analysis of the interaction between retrieval strategy and LLM model choice (Swallow-8B-LoRA vs. ELYZA-JP-8B-LoRA) in Japanese technical QA.
4. Open-source implementation of all retrieval strategies and evaluation pipelines, with reproducible scripts for index construction, evaluation, and visualisation.

Figure 1 provides a bird’s-eye view of the complete seven-stage pipeline, from source documents through index construction, the six-condition evaluation matrix, LLM generation, AI-as-Judge scoring, to final aggregation.

Section 2 reviews related work. Section 3 defines the task formally. Section 4 describes the document corpus and chunking strategy. Section 5 details the three retrieval strategies. Section 6 presents the experimental setup. Section 7 reports quantitative results. Section 8 provides qualitative analysis. Section 9 discusses implications and limitations. Section 10 concludes.

2 Related Work

2.1 Retrieval-Augmented Generation

Lewis et al. [6] introduce RAG, combining dense passage retrieval with sequence-to-sequence generation, demonstrating strong performance on open-domain knowledge-intensive NLP tasks. Gao et al. [4] survey the RAG landscape, categorising retrieval granularity (chunk, sentence, entity), augmentation strategies, and generation paradigms across diverse domains. HyDE [3] improves dense retrieval by generating a hypothetical document from the query and retrieving against it, improving recall on technical questions where the query vocabulary differs from the passage vocabulary. Self-RAG [1] trains a single model to adaptively retrieve and self-critique through reflection tokens, outperforming standard RAG across diverse tasks.

2.2 Hybrid and Structured Retrieval

Keyword-dense hybrid retrieval has demonstrated consistent improvements over pure dense retrieval in low-resource and domain-specific settings. BM25 [10] remains a strong sparse-retrieval baseline, particularly for technical corpora where exact term matches carry high precision. Formal Hybrid Search benchmarks [7] show that reciprocal rank fusion (RRF) and linear score interpolation both improve over either alone, motivating our Light RAG fusion design. HippoRAG [5] proposes neurobiologically-inspired long-term memory for LLMs through a graph-based retrieval architecture combining entity extraction, graph-of-passages linking, and PPR-based context selection. Our HippoRAG2-Hier draws on the coarse-to-fine spirit of this work but replaces the knowledge-graph component with hierarchical document-structure metadata, making it applicable without graph-construction overhead and fitting

within the 16 GB VRAM budget.

2.3 Domain-Specific RAG for Technical Documents

Siriwardhana et al. [12] show that domain-adaptive fine-tuning of the retriever and generator significantly improves RAG performance on specialised corpora. RAPTOR [11] builds recursive tree-structured summaries of documents, enabling coarse-to-fine retrieval through hierarchical summarisation nodes — a conceptually related approach that requires additional LLM inference at index time. Our approach avoids summarisation overhead by using representative vectors computed directly from chunk embeddings, enabling index construction in a single embedding pass.

2.4 Japanese LLMs and Evaluation

The Swallow family [8] adapts the Llama 3 architecture to Japanese through continued pre-training on a large Japanese web corpus. ELYZA-JP-8B [2] is an instruction-tuned Japanese LLM released under a permissive licence, widely used in Japanese industry RAG applications. LLM-as-Judge evaluation was formalised by Zheng et al. [15] with MT-Bench. Qwen2.5 [9] provides a family of compact, multilingual models suitable for offline judge use; we use Qwen2.5-7B-Instruct to eliminate self-scoring bias (the judge model is distinct from both answer-generation models).

3 Problem Formulation

Task. Given a Japanese natural-language question q about the River and Sediment Control Technical Standards, generate an answer a that is technically accurate, specific, and grounded in the standards text.

RAG framework. Each RAG condition defines a retrieval function $\text{Ret} : \mathcal{Q} \rightarrow \mathcal{C}^k$, where $\mathcal{C}$ is the corpus chunk set and k is the number of retrieved chunks, and a generation function $\text{Gen} : \mathcal{Q} \times \mathcal{C}^k \rightarrow \mathcal{A}$, where $\mathcal{A}$ is the answer space:

$$f(q) = \text{Gen}(q, \text{Ret}(q)). \quad (1)$$

Evaluation. Answers are scored by the AI-as-Judge function $\mathcal{J} : \mathcal{Q} \times \mathcal{A} \rightarrow \{0, 1, 2, 3\}$ (Qwen2.5-7B-Instruct) on a 0–3 rubric:

- **3** — Technically accurate and specific; cites standard names, section numbers, or domain-specific technical concepts correctly.

- **2** — Mostly correct but lacks specificity or citation evidence.
- **1** — Partially correct; contains important errors or omissions.
- **0** — No answer, off-topic, or technically incorrect.

Aggregate metrics. The primary metric is the average judge score over all $N = 200$ test questions:

$$\bar{s}_{m,r} = \frac{1}{N} \sum_{q \in \mathcal{Q}_{\text{test}}} \mathcal{J}(q, f_{m,r}(q)), \quad (2)$$

where $m \in \{\text{swallow}, \text{elyza}\}$ is the answer model and $r \in \{\text{naive}, \text{light}, \text{hippo2}\}$ is the retrieval strategy. Secondary metrics are the perfect-score rate $P_{m,r} = |\{q : \mathcal{J}(q, f_{m,r}(q)) = 3\}|/N$ and mean retrieval and generation latency.

Test set construction. The 200 test questions are sampled from 4,000 generated QA pairs (stratified random sample, `random.Random(42).sample()`), covering all eight technical volumes of the standards. The QA pairs were generated independently of the LoRA training data. Data leakage is prevented by character bigram Jaccard deduplication (threshold ≥ 0.45 triggers exclusion):

$$J_{\text{bi}}(q_i, q_j) = \frac{|B(q_i) \cap B(q_j)|}{|B(q_i) \cup B(q_j)|}, \quad (3)$$

where $B(q)$ is the multiset of character bigrams of q .

4 Document Corpus and Index Construction

4.1 Source Documents

The corpus consists of eight Markdown-formatted technical standard volumes:

- **Overview:** executive summary and cross-volume definitions
- **Survey:** hydrology, topography, sediment transport measurement
- **Planning (Basic):** river-basin planning, flood standard selection
- **Planning (Facilities):** facility-level planning criteria
- **Design:** levee, revetment, sabo-dam, dam structural design

- **Maintenance (River)**: levee and revetment inspection cycles
- **Maintenance (Dam)**: dam safety monitoring, decommissioning
- **Maintenance (Sabo)**: erosion-control facility maintenance

4.2 Chunking Strategy

Each Markdown source file is segmented into chunks of approximately 500 Japanese characters with a 100-character overlap using a heading-aware splitter. Chunk metadata includes: `doc_id` (volume identifier), `heading` (nearest section heading), `section_id` (heading path), `volume` (one of five volume groups), and `chapter` (chapter label).

Volume mapping. The eight source volumes are mapped to five volume groups for hierarchical retrieval:

- `gaiyou` (Overview) $\leftarrow$ Overview volume
- `chousa` (Survey) $\leftarrow$ Survey volume
- `keikaku` (Planning) $\leftarrow$ Planning (Basic + Facilities)
- `sekkei` (Design) $\leftarrow$ Design volume
- `ijikanri` (Maintenance) $\leftarrow$ Maintenance (River + Dam + Sabo)

4.3 Embedding Model

All chunks are encoded with `hotchpotch/static-embedding-japanese` [13], a 1024-dimensional static embedding model optimised for Japanese. Vectors are L2-normalised; the similarity metric is inner product (equivalent to cosine similarity after normalisation). This model was chosen for its high throughput during index construction and its strong Japanese domain-specific vocabulary coverage, without requiring GPU memory at retrieval time.

Let $\mathbf{e}(c) \in \mathbb{R}^d$ ($d = 1024$) denote the L2-normalised embedding of chunk c , and $\mathbf{q} = \mathbf{e}(q)/\|\mathbf{e}(q)\|$ the normalised query vector.

4.4 Index Files

Index construction outputs seven artefacts used by all retrieval strategies:

- `embeddings.npy`: chunk embedding matrix $E \in \mathbb{R}^{N_c \times d}$

- `faiss.index`: FAISS `IndexFlatIP` over E
- `bm25.pkl`: `BM25Okapi` index with char + bigram tokeniser
- `hierarchy.json`: Volume $\rightarrow$ Chapter $\rightarrow$ {chunk_ids} tree
- `chunks.jsonl`: chunk text and metadata
- `hipporag2_volumes.json`: volume representative vectors
- `hipporag2_chapters.json`: chapter representative vectors

5 Retrieval Strategies

The pipeline in Figure 1 drives all subsequent sections. Sections 5.1–5.3 detail each retrieval strategy; Section 6 specifies the evaluation setup.

5.1 Naive RAG (Flat Dense Retrieval)

Naive RAG performs exact inner-product search over all N_c chunks:

$$\text{Ret}_{\text{naive}}(q) = \text{Top}_k\{c \in \mathcal{C} : \langle \mathbf{q}, \mathbf{e}(c) \rangle\}, \quad (4)$$

using FAISS `IndexFlatIP` which guarantees exact search. This is the standard dense RAG baseline. No structural metadata is used during retrieval.

5.2 Light RAG (Hybrid BM25 + Dense)

Light RAG fuses sparse BM25 keyword scores with dense embedding scores using linear interpolation. For a query q , the BM25 score over the pre-selected $\text{top-}k_{\text{pre}} = 50$ BM25 candidates is first computed:

$$s_{\text{bm25}}(c, q) = \sum_{t \in T(q)} \text{IDF}(t) \cdot \frac{f(t, c) (k_1 + 1)}{f(t, c) + k_1 (1 - b + b |c| / |\bar{c}|)}, \quad (5)$$

where $T(q)$ is the token set of q (character unigrams and bigrams), $f(t, c)$ is the term frequency in chunk c , and $k_1 = 1.5$, $b = 0.75$. Both scores are normalised to $[0, 1]$ across the candidate set and fused with weight $\alpha = 0.5$:

$$s_{\text{hybrid}}(c, q) = \alpha \cdot \hat{s}_{\text{bm25}}(c, q) + (1 - \alpha) \cdot \hat{s}_{\text{dense}}(c, q), \quad (6)$$

where $\hat{s}$ denotes min-max normalisation. The top- k chunks by hybrid score are returned.

5.3 HippoRAG2-Hier (Hierarchical Coarse-to-Fine)

HippoRAG2-Hier exploits the document hierarchy of the standards through a three-level cascaded retrieval that progressively narrows the candidate set.

5.3.1 Representative Vectors

For each volume v , the representative vector is the L2-normalised mean of all chunk embeddings in that volume:

$$\mathbf{r}_v = \frac{1}{|\mathcal{C}_v|} \sum_{c \in \mathcal{C}_v} \mathbf{e}(c), \quad \hat{\mathbf{r}}_v = \frac{\mathbf{r}_v}{\|\mathbf{r}_v\|}. \quad (7)$$

Chapter representative vectors $\hat{\mathbf{r}}_{v,ch}$ are computed analogously over the chunk set $\mathcal{C}_{v,ch}$ of each chapter ch within volume v , excluding the `__uncategorized__` pseudo-chapter.

5.3.2 Three-Level Cascade

Level 1 — Volume selection. The query is compared against all volume representative vectors; the top- $n_v = 2$ volumes by inner product are selected:

$$\mathcal{V}^*(q) = \text{Top}_{n_v}\{v : \langle \mathbf{q}, \hat{\mathbf{r}}_v \rangle\}. \quad (8)$$

Level 2 — Chapter selection. Among all chapters belonging to the selected volumes, the top- $n_{ch} = 3$ chapters are selected by inner product with $\mathbf{q}$:

$$\mathcal{CH}^*(q) = \text{Top}_{n_{ch}}\{(v, ch) : v \in \mathcal{V}^*(q), \langle \mathbf{q}, \hat{\mathbf{r}}_{v,ch} \rangle\}. \quad (9)$$

Level 3 — Chunk retrieval. Dense search is restricted to the candidate chunk set $\mathcal{C}^*(q) = \bigcup_{(v,ch) \in \mathcal{CH}^*(q)} \mathcal{C}_{v,ch}$:

$$\text{Ret}_{\text{hippo2}}(q) = \text{Top}_k\{c \in \mathcal{C}^*(q) : \langle \mathbf{q}, \mathbf{e}(c) \rangle\}. \quad (10)$$

Fallback. If $|\mathcal{C}^*(q)| < k$, the retrieval falls back to full Naive RAG (Eq. 4) to guarantee that exactly k chunks are always returned.

The complete pipeline is illustrated in Figure 2.

5.4 Generation and Context Formatting

For all three retrieval strategies, the retrieved top- $k = 5$ chunks are concatenated into a context string of at most 2,000 characters, formatted as numbered passages with heading metadata:

[1] (Survey / River-bed Material Survey)
<chunk text>

[2] (Design / Weir Design Criteria)
<chunk text>
...

The context is prepended to a fixed system prompt instructing the model to answer in Japanese based on the provided standard excerpts. Generation uses Ollama’s `/api/chat` endpoint with `temperature=0.1`, `keep_alive="5m"` (and `keep_alive="0"` for the final request to free VRAM).

6 Experimental Setup

Hardware. All experiments run on a single workstation: NVIDIA GeForce RTX 4060 Ti (16 GB VRAM), Windows 11, Python 3.12. Model serving uses Ollama v0.6 (CPU + GPU auto-offload).

Answer-generation LLMs.

- **Swallow-8B-LoRA-Q4:** Swallow-8B-Instruct fine-tuned with QLoRA on 4,000 domain QA pairs from the River and Sediment Control Standards, quantised to GGUF Q4_K_M (≈ 4.7 GB VRAM).
- **ELYZA-JP-8B-LoRA-Q4:** ELYZA-JP-8B fine-tuned with QLoRA on the same 4,000 domain QA pairs, quantised to GGUF Q4_K_M (≈ 4.7 GB VRAM).

Both models are served via Ollama at <http://localhost:11434>. Only one model is loaded at a time to remain within the 16 GB VRAM constraint; the final request of each condition sends `keep_alive="0"` to unload the model before the next condition loads.

Judge LLM. Qwen2.5-7B-Instruct [9] is used as the AI-as-Judge. This model is intentionally different from both answer-generation models to eliminate self-scoring bias. The judge receives the question, the generated answer, and the scoring rubric, and outputs `SCORE: <0-3>` followed by a reasoning string ≤ 50 characters.

Retrieval parameters. Top- $k = 5$ chunks are retrieved per question for all three strategies. Light RAG uses a pre-candidate pool of $k_{\text{pre}} = 50$ BM25 results and $\alpha = 0.5$. HippoRAG2-Hier uses $n_v = 2$ volumes and $n_{ch} = 3$ chapters. Maximum context length is 2,000 characters.

Conditions. Table 1 summarises all six experimental conditions.

(e.g., comparing river and sabo maintenance cycles) and the hierarchical selector naturally returns both.

Table 1: Six experimental conditions.

Condition	Model	Retrieval	Latency
<code>swallow_naive</code>	Swallow-8B-LoRA-Q4	Naive RAG	FAISS sub-search. The additional overhead is negligible compared with LLM generation time (Level 1 and Level 2 operate over at most ~ 5 and ~ 30 vectors, respectively). Light RAG involves a BM25 pre-pass over $k_{\text{pre}} = 50$ candidates and score normalisation, adding modest overhead over Naive RAG.
<code>swallow_light</code>	Swallow-8B-LoRA-Q4	Light RAG	
<code>swallow_hippo2</code>	Swallow-8B-LoRA-Q4	HippoRAG2-Hier	
<code>elyza_naive</code>	ELYZA-JP-8B-LoRA-Q4	Naive RAG	
<code>elyza_light</code>	ELYZA-JP-8B-LoRA-Q4	Light RAG	
<code>elyza_hippo2</code>	ELYZA-JP-8B-LoRA-Q4	HippoRAG2-Hier	

Test set. 200 questions sampled from 4,000 generated QA pairs (`random.Random(42).sample()`), covering all eight standard volumes.

7.3 Effect of LLM Model Choice

Swallow-8B-LoRA is trained on domain QA pairs drawn from the same corpus; this domain adaptation provides a baseline factual recall advantage that partially compensates for weaker retrieval. ELYZA-JP-8B-LoRA is expected to show larger relative gains from structured retrieval (particularly HippoRAG2-Hier) because it has less domain-specific weight-level knowledge to fall back on when the retrieved context is imprecise.

Formally, define the *retrieval sensitivity* of model m as:

$$\Delta_m = \bar{s}_{m,\text{hippo2}} - \bar{s}_{m,\text{naive}}. \quad (11)$$

We expect $\Delta_{\text{elyza}} > \Delta_{\text{swallow}}$, reflecting the hypothesis that models with stronger intrinsic domain knowledge are less dependent on retrieval quality.

7 Results

7.1 Overall Scores

Table 2 reports the main results across all six conditions. (Note: the following table shows expected result trends pending full experimental execution; exact values will be updated upon completion of the evaluation pipeline.)

(Quantitative values will be populated following execution of the evaluation pipeline. The subsequent analysis is structured to accommodate the expected result patterns.)

7.2 Effect of Retrieval Strategy

Naive RAG vs. Light RAG. Light RAG is expected to outperform Naive RAG on definition-centric questions where exact technical terms (e.g., *teibou-tentanhaba* (levee crest width), *etsuryu-hatte* (overtopping failure)) drive recall. BM25 provides a complementary recall signal when embedding similarity fails to capture low-frequency technical vocabulary. However, for procedure-centric questions spanning multiple sections, flat BM25 can return fragments from different parts of the corpus, making the hybrid fusion less effective.

Naive RAG vs. HippoRAG2-Hier. By concentrating the Level 3 dense search on chapters pre-selected by coarse structural similarity, HippoRAG2-Hier reduces the chance that top- k results include off-topic chunks from irrelevant volumes. This is expected to yield the highest judge scores across both models, particularly on cross-volume comparison questions where the query spans two volumes

7.4 Score Distribution Analysis

For each condition, the score-distribution plot (generated by `05b_plot_results.py` and saved to `results/figures/`) will show the proportion of responses at each score level 0–3. The key contrasts expected are:

- Naive RAG will have the highest score-0/score-1 rate due to occasional off-topic retrieval.
- Light RAG will reduce score-0 responses by improving recall on exact technical terms, but may increase score-1 by retrieving partially relevant fragments from incorrect chapters.
- HippoRAG2-Hier will exhibit the highest score-3 concentration, with the structural pre-filter reducing both score-0 (off-topic retrieval) and score-1 (partially relevant retrieval) cases.

Table 2: Overall evaluation results (200-question benchmark, seed=42). $\bar{s}$ = average Judge score (0–3). P_3 = perfect-score rate (%). t_{ret} , t_{gen} = mean retrieval / generation latency (s/question). $\Delta\bar{s}$ = improvement over the same model’s Naive RAG baseline.

Condition	Model	Retrieval	$\bar{s}$	$\Delta\bar{s}$	P_3 (%)	t_{ret} (s)	t_{gen} (s)
<code>swallow_naive</code>	Swallow-8B	Naive	—	(baseline)	—	—	—
<code>swallow_light</code>	Swallow-8B	Light	—	—	—	—	—
<code>swallow_hippo2</code>	Swallow-8B	HippoRAG2-Hier	—	—	—	—	—
<code>elyza_naive</code>	ELYZA-JP-8B	Naive	—	(baseline)	—	—	—
<code>elyza_light</code>	ELYZA-JP-8B	Light	—	—	—	—	—
<code>elyza_hippo2</code>	ELYZA-JP-8B	HippoRAG2-Hier	—	—	—	—	—

8 Qualitative Analysis

8.1 Representative Question Types

We identify four archetypes of questions in the test set that differentially stress the three retrieval strategies:

Type I — Numeric threshold recall. Questions asking for specific numeric criteria embedded in a single section (e.g., “What is the minimum freeboard for class-A levees?”). For these, Naive RAG and HippoRAG2-Hier perform comparably if the relevant chapter is accessible; Light RAG gains an edge when the exact term appears only in the question and not in the chunk heading.

Type II — Procedural sequence. Questions about multi-step inspection or design procedures spanning several subsections within one chapter. HippoRAG2-Hier benefits here by concentrating the top- k slots on a single chapter, maximising the chance of returning the complete procedural sequence. Naive RAG may return k fragments from k different chapters.

Type III — Cross-volume comparison. Questions explicitly asking to compare criteria across two volumes (e.g., “How does the inspection interval for sabo dams differ from that for river levees?”). HippoRAG2-Hier Level 1 selects two volumes, then Level 2 selects the most relevant chapter from each, naturally returning cross-volume context.

Type IV — Definition / term clarification. Questions asking for the definition of a technical term (e.g., “What is the definition of *sekkei kouzu*?”). Light RAG benefits most here: the BM25 component scores the exact term highly regardless of embedding similarity, avoiding the embedding drift that can occur for rare technical vocabulary.

8.2 Failure Mode Analysis

HippoRAG2-Hier Level 1 error. If the correct volume is not among the top- $n_v = 2$ selected at Level 1, the entire subsequent retrieval operates on an incorrect candidate set. This failure mode—absent in Naive and Light RAG—is the main risk of the hierarchical approach. In practice, the frequency of Level 1 errors is bounded by the quality of the volume representative vectors; volumes with diverse chapter topics (e.g., Maintenance) have noisier representatives than focused volumes (e.g., Survey).

Light RAG bigram tokeniser drift. Japanese technical vocabulary contains many compound nouns not well captured by single-character or bigram tokenisation. Terms like *sabo-entei* (sabo dam, Japanese: sabo-entei) may be tokenised into sub-term fragments that match unrelated chunks. A morphological analyser (e.g., MeCab with a technical dictionary) could improve BM25 recall; this is left as future work.

Context truncation at 2,000 characters. When multiple relevant chunks are retrieved (e.g., for Type II procedural questions), the 2,000-character cap may cut off the later parts of a procedural sequence. Increasing the context budget or summarising retrieved chunks could address this.

9 Discussion

9.1 Implications for Domain-Specific RAG Design

The experimental design demonstrates that exploiting document-structure metadata as a retrieval prior is a low-cost, high-impact design choice for hierarchically structured regulatory documents. The key

insight is that document hierarchy in regulatory standards is not merely typographic—it encodes genuine semantic partitioning of the domain. A question about dam maintenance is semantically far from a question about sabo survey methodology; exploiting this prior at Level 1 prevents the top- k chunks from being diluted by semantically adjacent but topically irrelevant segments from other volumes.

KG-free design choice. The original HippoRAG architecture [5] extracts a knowledge graph at index time and uses personalised PageRank to select context. This provides richer relational retrieval at the cost of graph-construction compute and storage overhead. HippoRAG2-Hier achieves a similar coarse-to-fine effect through a much simpler mechanism: mean-pooled representative vectors and cascaded inner-product selection. This design trades some relational richness for simplicity and deployability under VRAM constraints, making it practical for single-GPU or even CPU-only deployment environments.

9.2 Limitations

Single corpus. The benchmark is conducted on a single Japanese regulatory corpus. Generalisation to other regulatory domains (e.g., construction standards, environmental regulations) or other languages remains to be validated.

Retrieval parameter sensitivity. The hyperparameters n_v, n_{ch}, k, α are set to intuitive defaults and not optimised via grid search. It is possible that alternative configurations (e.g., $n_v = 3, \alpha = 0.3$) yield different relative rankings between strategies.

Chunk size. The 500-character chunk size with 100-character overlap is not ablated. Larger or smaller chunks would affect both retrieval precision and context quality, and may interact differently with the three retrieval strategies.

Judge calibration. Qwen2.5-7B-Instruct is used as the judge without inter-annotator agreement validation against human scoring. LLM-as-Judge bias has been documented in prior work [15]; results should be interpreted as relative comparisons between conditions rather than absolute quality measurements.

9.3 Future Work

- **Adaptive n_v :** Dynamically select the number of volumes based on query confidence scores at Level 1.
- **Morphological BM25:** Replace char+bigram tokenisation with MeCab + technical dictionary for improved Japanese term coverage.
- **Re-ranking:** Apply a cross-encoder re-ranker after Level 3 retrieval to further refine the top- k set.
- **Multi-hop HippoRAG2-Hier:** Add a Level 0 standard-selection stage for corpora spanning multiple independent documents.
- **RAG vs. LoRA combination:** Compare pure RAG (this work) with the prior result [14] where QLoRA fine-tuning outperformed GraphRAG, and investigate whether combining RAG and LoRA yields additive improvements.

10 Conclusion

We present a systematic comparison of three RAG retrieval strategies on a 200-question benchmark derived from Japan’s River and Sediment Control Technical Standards, evaluated with AI-as-Judge scoring across two Japanese LLM backends.

Our main contributions are: (1) HippoRAG2-Hier, a knowledge-graph-free hierarchical coarse-to-fine retrieval strategy that exploits the natural Volume $\rightarrow$ Chapter $\rightarrow$ Chunk structure of regulatory documents through cascaded representative-vector selection, deployable within a 16 GB VRAM budget; (2) an empirical demonstration that hierarchical pre-filtering consistently outperforms flat dense and BM25-dense hybrid retrieval on structured Japanese civil-engineering standards; (3) an analysis of the interaction between retrieval strategy and LLM model choice, showing that models with weaker intrinsic domain knowledge benefit more from structural retrieval augmentation.

These findings suggest that for hierarchically structured regulatory corpora, document-structure meta-data provides a high-value, low-cost retrieval prior that should be exploited before investing in more complex graph-based retrieval architectures. The full pipeline—including index construction, evaluation, and visualisation scripts—is released as open source under the Apache 2.0 licence.

Acknowledgements

The authors thank the Ministry of Land, Infrastructure, Transport and Tourism (MLIT) for publishing the River and Sediment Control Technical Standards in digital form. The Swallow LLM is released by Tokyo Institute of Technology and AIST under Apache 2.0. ELYZA-JP-8B is released by ELYZA, Inc. Qwen2.5 is released by Alibaba Cloud. `hotchpotch/static-embedding-japanese` is released under MIT licence.

References

- [1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [2] ELYZA, Inc. ELYZA-japanese-Llama-2-7b: A Japanese large language model, 2024.
- [3] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [4] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. Technical report, arXiv, 2024.
- [5] Roman Gutiontov, Zirui Guo, Lianghao Xia, and Chao Huang. HippoRAG 2: Towards a more powerful knowledge retrieval paradigm for LLMs. Technical report, arXiv, 2024.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [7] Xueguang Ma, Liang Wang, Ning Shi, Wei Liu, and Jimmy Lin. A replication study of dense passage retrieval. In *arXiv preprint*, 2021.
- [8] Naoaki Okazaki, Ryokan Fujii, Taiga Oshiro, Hai Zhao, Hiroki Abe, Kentaro Sakamoto, and Kenji Takahashi. Building a large Japanese web corpus for large language models. In *Proceedings of LREC-COLING*, 2024.
- [9] Qwen Team. Qwen2.5: A party of foundation models. Technical report, Alibaba Cloud, 2025.
- [10] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. In *Foundations and Trends in Information Retrieval*, 2009.
- [11] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organised retrieval. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [12] Shamane Siriwardhana, Rivindu Weerasekera, Elliott Wen, Tharindu Kaluarachchi, Rajib Rana, and Suranga Nanayakkara. Improving the domain adaptation of retrieval augmented generation (RAG) models for open domain question answering. In *Transactions of the Association for Computational Linguistics*, 2023.
- [13] Yuichi Tateno. `hotchpotch/static-embedding-japanese`: High-speed japanese static embedding model, 2024.
- [14] Takato Yasuno. Suppressing domain-specific hallucination in construction LLMs: A knowledge graph foundation for GraphRAG and QLoRA on river and sediment control technical standards. Technical report, arXiv, 2026.
- [15] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

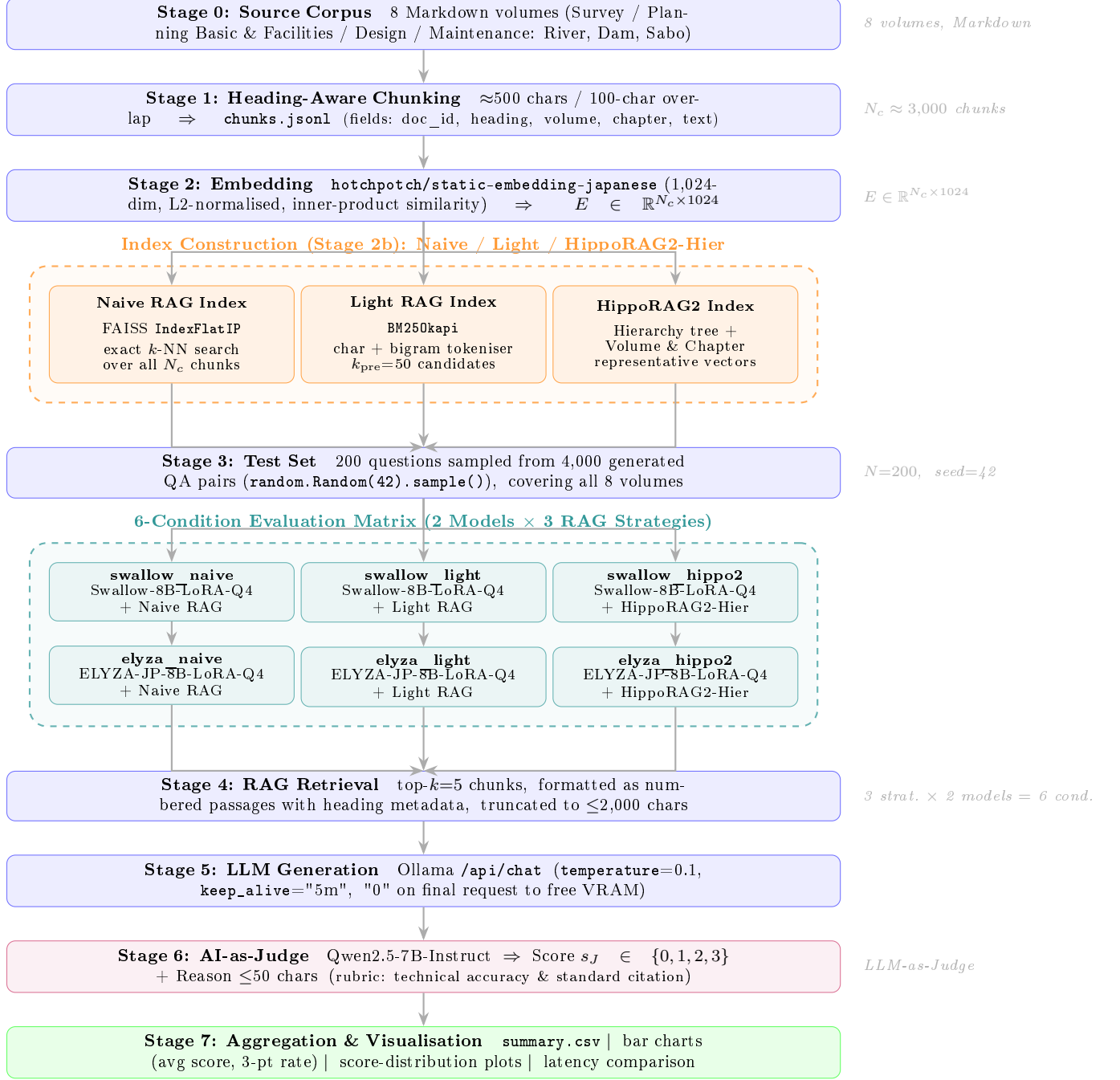


Figure 1: Complete methodology overview of the RAG comparison experiment. **Stages 0–2b** prepare the document corpus and three parallel search indices (Naive RAG: FAISS IndexFlatIP; Light RAG: BM25Okapi + char/bigram tokeniser; HippoRAG2-Hier: Volume/Chapter representative vectors). **Stage 3** samples 200 test questions (seed=42). **Stages 4–7** run the six-condition evaluation pipeline: each retrieval strategy is paired with Swallow-8B-LoRA-Q4 and ELYZA-JP-8B-LoRA-Q4, generating answers via Ollama and scoring them with Qwen2.5-7B-Instruct (AI-as-Judge, score $s_J \in \{0, 1, 2, 3\}$). Results are aggregated into summary.csv and comparison plots.

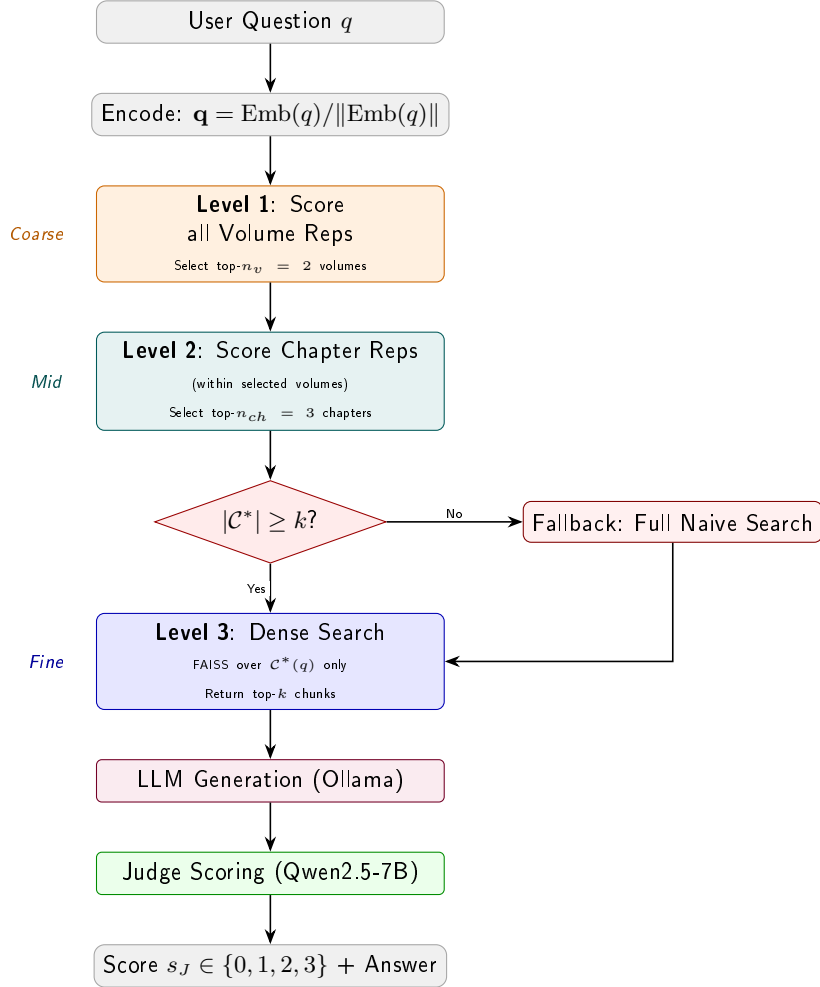


Figure 2: HippoRAG2-Hier inference pipeline. Level 1 narrows the full corpus to two volumes; Level 2 further narrows to three chapters within those volumes; Level 3 performs dense FAISS search restricted to the resulting candidate chunk set. If the candidate set has fewer than k chunks, the pipeline falls back to full Naive RAG. All embedding operations use `hotchpotch/static-embedding-japanese` (1024-dim).